

CS F213 OOP

Project: Digital ecosystem of annual cultural festival management system

Problem Statement:

As a student developer at BITS Pilani, you are responsible for maintaining and enhancing the digital ecosystem of the annual cultural festival serving 5,000–6,000 users with real-time operations including ticketing, food ordering, digital wallet (₹1.5+ crore transactions), event management, vendor coordination, accommodation, and live dashboards. The backend is a Django monolith (PostgreSQL, Redis, Celery, WebSockets), and the mobile app is the primary user interface.

Several operational bottlenecks and opportunities for intelligent automation have been identified. You are free to work on any one of the following problem tracks. Each track demands a different set of Java skills (as per the handout) and must be justified in your report.

This assignment builds a large-scale festival management platform suitable for 5,000–6,000 users conducting ₹1.5+ crore transactions with support for real-time operations. Choose one problem from a broad set of categories (mathematical modelling, AI algorithms, data structures & algorithms, or Java framework-based application development) and implement a software component that integrates with the festival management ecosystem. You must apply key Java concepts such as collections, concurrency, generics, exception handling, file I/O, serialisation, inner classes, etc. to ensure a robust, production-quality solution.

General Objectives

- Solve a real-world problem derived from operational constraints.
- Demonstrate advanced Java programming skills beyond basic syntax.
- Integrate your solution with existing solution architecture modules (e.g., Event Management System, food ordering, wallet, ticketing, vendor system).
- Produce a well-documented, testable, and extensible component.

1. Mathematical Models for Operations

Example Usecase: 90+ events are scheduled across multiple venues with limited time slots. Conflicts arise when a participant wishes to attend two overlapping events, or when the same resource (venue, judge, equipment) is double-booked. Currently, scheduling is done manually and often results in clashes, participant complaints, and inefficient resource usage.

Formulate and implement a mathematical model (e.g., linear programming, constraint programming, game theory, or discrete event simulation) to improve a specific application workflow.

Example problems:

- Event scheduling with resource constraints – model as integer linear programming; minimise conflicts subject to venue, staff, and time limits.
- Dynamic pricing for professional shows – demand-based pricing model using historical ticket sales data.
- Food queue length prediction – queuing theory model to predict waiting times and recommend additional vendor counters.
- Accommodation allocation – bipartite matching or Hungarian algorithm for assigning outstation participants to hostel rooms.

Expected Java features: `java.util` collections, custom exceptions, file I/O for input data, possibly `java.util.concurrent` for simulation steps.

2. Computational Algorithms in Artificial Intelligence

Example Usecase: Participants often miss exciting events because they are overwhelmed by the 90+ options. The current mobile app shows a flat list of events with no personalisation. Meanwhile, the system logs user browsing history, past registrations, and wallet spending patterns.

Design and implement an AI-driven component that adds intelligent behaviour to the platform.

Example problems:

- Personalised event recommendation engine – collaborative filtering or content-based filtering using participant browsing history, past registrations.
- Real-time anomaly detection in wallet transactions – simple rule-based or clustering to flag suspicious payments.
- Chatbot / RAG assistant for participant queries – use a lightweight in-memory vector store and a retrieval-augmented generation pipeline that is either simulated or with a local LLM API.
- Crowd flow prediction during peak hours – time-series forecasting (ARIMA or LSTM simulation) based on historical entry/exit logs.

Expected Java features: Generics (`List<Transaction>`, `Map<User, List<Event>>`), custom comparators, thread pools for batch processing, serialisation to save trained models (or model parameters), inner classes for data holders.

3. Problem Solving with Algorithms and Data Structures

Example Usecase: During multi-stage competitions (e.g., dance, debate, quiz), judges enter scores via tablets. The current system updates a leaderboard but does so inefficiently – recalculating ranks from scratch each time, causing lag when many scores arrive concurrently (e.g., during finals).

Implement a non-trivial algorithm or data structure to solve a computational bottleneck.

Example problems:

- Clash-free timetable generator – backtracking with constraint propagation (CSP) for 90+ events.
- Optimised ticket allocation – priority queue (max-heap) for waitlist management and dynamic seat assignment.
- Shortest path for volunteer dispatch – Dijkstra / A* on a graph of venue locations, integrated with the vendor ordering system.
- Real-time leaderboard for competitions – balanced binary search tree (or TreeMap) to maintain top scores with $O(\log n)$ updates and $O(k)$ retrieval.
- Efficient search for region-specific local products – inverted index or trie for fast keyword filtering.

Expected Java features: Custom data structures (or advanced use of java.util – TreeMap, PriorityQueue, HashMap), recursion, algorithm visualisation (optional), exception handling for edge cases (empty data, invalid input), file I/O to load initial data.

4. Application Development with Java Frameworks

Example Usecase: The food vendors struggle with order overload during peak hours. The current vendor app sometimes lags. We need a lightweight desktop order dashboard that vendors can run locally. The dashboard must display incoming orders in real-time, allow status updates (preparing → ready → served), and maintain an offline cache for network failures.

Build a functional module that extends the platform using a recognised Java framework or core Java with advanced features.

Example problems:

- Vendor order dashboard (desktop GUI) – AWT/Swing application displaying real-time food orders with Observer pattern and multithreading.
- Offline ticket validation tool – Standalone app caching ticket data via serialisation and validating QR codes without network.
- Food ordering simulation & load testing – Multi-threaded client simulator generating user load on food ordering API.
- JDBC-based wallet transaction logger – Database application logging all wallet transactions and providing an admin query interface.
- Pattern matching utility for event search – GUI tool using regular expressions to search event descriptions and highlight matches.

Expected Java features: Multithreading (Thread, ExecutorService, synchronisation, CountdownLatch, SwingWorker), GUI (AWT, Swing, JFrame, JTable, event listeners, adapters), collections (ArrayList, HashMap, TreeMap, PriorityQueue, ConcurrentHashMap), I/O & serialisation (FileInputStream, ObjectOutputStream, BufferedReader, try-with-resources), JDBC (DriverManager, Connection, PreparedStatement, ResultSet, SQLException), lambda expressions (functional interfaces, action listeners, comparators, Runnable), regular expressions (Pattern, Matcher, PatternSyntaxException), design patterns (Observer, MVC, DAO, Factory, Singleton, Producer-Consumer), OOP (inheritance, polymorphism, generics, custom exceptions, inner classes).

Assignment Tasks & Deliverables

Part A - Problem Formulation: A short presentation (≤ 10 slides) that precisely defines the problem, inputs/outputs, success metrics, innovation and integration with specific modules. Must explain which Java features will be used and why they are appropriate. This evaluation component must be submitted before the mid-semester examination. 5 Marks.

Part B - Problem Solution: Complete implementation in Java (or Java-based framework). Must compile and run, demonstrate OOP principles in design, use at least two advanced Java concepts (e.g., collections, generics, multithreading, exception handling, file I/O, serialisation, inner classes, etc.). Provide unit tests to support correctness, a demo video (15-20 minutes), and integration instructions in documentation. This evaluation component must be submitted before the end-semester examination. 5 Marks.

General Constraints (for all tracks)

- Integration with existing system: Demonstrate interfaces with existing modules. This can be via:
 - Mock REST APIs (simulate using simple HTTP stubs or file-based data).
 - Direct file exchange (e.g., reading/writing CSV or JSON that the backend can produce/consume).
 - JDBC connection to a shared database (if you set up a local schema mirroring database tables).
 - Simulated real-time streams (using BlockingQueue to mimic WebSocket or Firebase messages).
- Documentation: Your report must include:
 - Problem definition (inputs, outputs, success metrics).
 - Explanation of chosen Java features (refer to syllabus weeks).
 - Instructions to run the program and integrate with a simulated environment.
 - Submit PDF report, code repository (GitHub/zip), and demo video.
- Demo Video (15–20 minutes): Show your working application, explain each module, and demonstrate integration (e.g., reading a sample event file, responding to mock order requests, updating a simulated database).

Evaluation Rubrics:

Part A: Problem Formulation (5 marks)

Criteria	Excellent (5)	Good (4)	Satisfactory (3)	Needs Improvement (≤ 2)
Clarity & completeness	Problem precisely defined; inputs, outputs, constraints, success metrics fully described.	Minor missing details.	Some ambiguity or missing elements.	Vague or incomplete definition.
Integration with existing platform	Clear mapping to specific platform modules (e.g., event management, wallet, vendor system). Realistic API or data flow described.	Integration described but lacks specificity.	Integration mentioned superficially.	No meaningful integration with the platform.
Feasibility & scope	Appropriately scoped for 2-3 weeks; realistic constraints.	Slightly too large or small but doable.	Scope issues that would hinder completion.	Infeasible or irrelevant problems.
Use of advanced Java features	Clearly justifies at least two advanced features (collections, concurrency, JDBC, serialisation, lambdas, regex, design patterns,	Justifies one advanced feature; others mentioned.	Weak connection to Java advanced topics.	No justification.

	etc.) with specific examples.			
Innovation / originality	Novel approach to a real operational pain point; shows insight into system workflows.	Some interesting ideas.	Standard textbook formulation.	Trivial or copied formulation.

Part B: Problem Solution (5 marks)

Criteria	Excellent (5)	Good (4)	Satisfactory (3)	Needs Improvement (≤ 2)
Correctness & functionality	Solution correctly solves the formulated problem; handles edge cases; integrates with mock/real backend endpoints.	Mostly correct; minor bugs or integration issues.	Partial functionality; major gaps.	Does not work or fails core requirements.
OOP design & code quality	Clean, modular, extensible; appropriate use of classes, inheritance, polymorphism, and at least one design pattern. Readable with comments.	Good OOP but missing some patterns or slightly messy.	Poor separation of concerns, procedural style.	No OOP structure.
Integration with platform	Successfully interfaces with the target platform (REST, WebSocket, file-based, JDBC, or well-defined	Integration works with minor flaws.	Partial or fragile integration.	No real integration.

	stubs). Data flow matches formulation.			
Use of advanced Java features	Correctly implements two or more advanced features from the syllabus (e.g., multithreading, JDBC, serialisation, collections framework, generics, lambdas, regex, design patterns). Demonstrates understanding.	Implements one advanced feature correctly; the second is trivial or flawed.	Advanced features attempted but incorrect.	No advanced Java features.
Testing & documentation	Comprehensive unit tests; clear README, integration instructions, and a demo video (5-10 min) showing live integration.	Tests cover main paths; documentation mostly clear.	Minimal testing or poor documentation.	No tests or documentation.

Submission Guidelines

- Report (PDF) – problem formulation (≤15 pages) + solution description + UML class diagram (if applicable).
- Code (GitHub or zip) – well-structured, with README, build/run instructions, and unit tests.
- Demo video (15–20 minutes) – each group member introduces themselves and explains a module. Show integration with realistic mocks.
- Deadline – as per course policy (hard deadline). No extensions.

Important Note:

- Students work in groups (which should have been formed). The evaluation will be done for the group and all the members will get the same score. It will be up to the members to have a fair share of contribution for the successful implementation/development of the project. Arguments or feedback on individual member's contributions will not be entertained. If someone is not giving a valuable contribution, then it depends upon other members to decide how to engage with the non-performing member. IC will not indulge.
- You are free to design the GUI for the above project as per your creativity. Appropriate links should be provided to transit from one option to another. Differences in GUI need to be maintained for both user and delivery person. Feel free to make your assumptions in order to make the implementation better. Not everything can be mentioned in the description.
- You are not expected to develop from scratch. You can feel free to make use of available APIs or packages, AI tools or techniques, etc. The idea is to expose you to project development experience. High use of modern advanced technology, platform, and paradigm will definitely attract high grades. The technology stack should be mentioned along with its usage in the project report.
- The basic functionalities are mandatory for project evaluation. A group is free to add more innovative functionalities which helps in enhancement of the problem statement, and it will carry additional marks.
- The marking will purely depend upon the evaluators and their evaluation will be final. No arguments in this regard will be entertained and will attract negative marking as per IC's discretion. In case of unethical practices, if observed, or brought to my notice, then without any discussion (as cannot engage in forensic investigations nor act as a moral police) all the groups involved will be awarded 0 marks.
- A working software is expected, if your code will not execute then it will not be evaluated and some default mark will be awarded. Each group needs to make a PDF report and record a video (demo of working application) presentation (15-20 minutes) of the developed application involving all the group members and upload it to a shared folder (link to upload will be shared later). The video file should be named by the Group No. Supposing your Group No. is 1, then the file should be named as 1_OOP_Summer_2026_Project. While recording the demo (working application), each member should introduce himself/herself and explain the functioning of any module(s).
- The last date to upload the recorded video presentation and the report will be posted in GC. This is a hard deadline and no request to extend the deadline will be entertained for operational/official limitations.
- Do good work so that you will be able to reflect on your resume and feel proud of your efforts.